

Projekt do předmětu ISA

Monitoring SSL spojení

Fakulta informačních technologií
Vysoké učení technické v Brně

Autor: Luděk Burda
login: xburda13

Datum: 18.11.2020

Obsah

Úvod.....	3
Implementační detaily.....	4
Testování a zdroje.....	6

Úvod

Cílem projektu je vytvořit aplikaci pro zobrazení informací o SSL spojení. Tato spojení program buď zachytává v reálném čase na určitém rozhraní, nebo je získá analýzou pcapng souboru, jenž slouží k uložení proběhlé síťové komunikace.

SSL (novější, aktuálně rozšířená verze je TLS) komunikace probíhá na transportní vrstvě TCP, před každou komunikací musí proběhnout tzv. 3 way handshake, díky němuž naváže komunikace klient se serverem. Následně klient odešle server TLS handshake typu Hello a počká na přijaté Hello od serveru. Klient se serverem si vymění informace o použité šifře a vygenerují klíče pro rozšifrování komunikace. Od této chvíle je SSL spojení navázáno a následuje výměna dat. Spojení je poté ukončeno TCP zprávou s příznakem FIN, případně RST. V některých případech po odeslání FIN zprávy klientem ještě ze serveru přijdou nějaká data, některé aplikace očekávají potvrzení o ukončení spojení od serveru, některé ne. V mém programu jsem se rozhodl považovat spojení za ukončené až po potvrzující FIN zprávě od serveru.

Program je implementován v jazyce C, jenž vyniká v přímé komunikaci s hardwarem a svojí rychlostí. Návod pro spuštění programu se nachází v manuálové stránce `sslsniff.1`. V případě spuštění s příznakem `-i` je nezbytně nutné provést spuštění s oprávněními správce.

Implementace dvou požadovaných funkcionalit se v mém případě výrazně liší, informace o implementaci jsou popsány v kapitole 2 Implementační detaily.

Aplikace je schopná analyzovat pouze PCAPNG soubory, nikoliv PCAP soubory a je schopná zpracovávat pouze pakety z ethernetové linkové vrstvy. Pro výstup potřebný počet paketů počítá od zahájení TCP handshaku, po oboustranné ukončení TCP komunikace.

Implementační detaily

sslsniff.c

Hlavní soubor jenž pouze volá ostatní moduly.

parseArguments.c (parseArguments.h)

Modul a pro zpracování argumentů.

sniffer.c (sniffer.h)

Modul sloužící k zachytávání komunikace v reálném čase. V hlavní funkci modulu jsou nejdříve připraveny objekty potřebné ke správné funkčnosti sniff relace. Tato pasáž je zdokumentována v samotném kódu. Poté následuje část společná jak pro modul **sniffer.c** tak **pcapanal.c**, tato část bude popsána dále v textu.

pcapanal.c (pcapanal.h)

Modul, v němž probíhá analýza PCAPNG souborů a následné zpracování jejich obsahu. Na začátku je načten soubor binárně a uložen do pole bytů. PCAPNG soubor je rozdělen do bloků zarovnaných na 32 bitových hodnotách. Podle standardu by měl první blok odpovídat Section Header Blocku. Tento blok je identifikován palindromickým hexadecimálním číslem 0x0A0D0D0A, tedy je možné ho rozpoznat i bez znalosti endianity. Od 9. po 12. byte včetně je oblast kde se nachází tzv. Magic number. Podle tohoto čísla lze určit endianitu všech vícebytových částí bloků, z nichž je soubor tvořen. Pro tento účel jsem vytvořil jednoduché makro, jenž přijímá hodnotu a podle nastavení proměnné inicializované na základě výsledku analýzy Magic number rozhodne, zda s hodnotou neudělá nic, nebo jí změní endianitu.

Následuje hlavní cyklus, který běží dokud není přečtené celé pole bytů se souborem. Na začátku je vždy zkontrolován typ bloku. Typicky se zde nachází převážně Enhanced Packet Blocky, v nichž je uložen samotný obsah paketů s režijními informacemi.

Vyjma tohoto bloku je zde ale také Interface Description Block, ve kterém se nachází klíčová informace podstatná pro získání timestampu jednotlivých paketů. Tato informace je uchována v oblasti Options a definuje, v jakých časových jednotkách je uložen epocha time jednotlivých paketů daného interface popsaného tímto blokem.

V samotných blocích Enhanced Packet Block je mimo samotný obsah packetu vždy uchována již zmíněná 64 bitová hodnota timestamp rozdělená do dvou 32 bitových slov. Tyto 32 bitová slova je nutné načíst zvlášť a například pomocí bitových operací je překonvertovat na jednu 64 bitovou hodnotu. Tuto hodnotu je následně potřeba naškálovat pomocí informace získané v IDB. Poté již máme epocha time v sekundách a můžeme tuto hodnotu předat funkci, jenž nám čas převede do čitelného formátu.

Společná část pcapanal.c a sniffer.c

Předpokládáme, že máme připravený ukazatel na začátek packetu. Následující proces je vykonán jak při offline, tak při online analýze komunikace. Nejvíce vnější vrstva packetu je ethernetový rámec, zde se nachází pouze jedna podstatná informace, a tou je verze IP uložená na 13. a 14. bytu. Dále posuneme buffer o 14 bytů, což je fixní délka daného rámce.

Poté se ukazatel nachází na začátku IP hlavičky. Zde existuje odlišnost mezi IPv4 a IPv6 hlavičkou. Primárním rozdílem je velikost IP hlavičky, jenž je u IPv6 fixních 40 bytů, u IPv4 hlavičky je potřeba tuto informaci získat z nižší čtveřice prvního bytu hlavičky. Dále z IP hlavičky získáme pouze IP adresy odesílatele a příjemce, jenž mají v obou typech hlavičky odlišnou polohu i velikost, po zjištění, zda je transportním protokolem námi požadované TCP, inkrementujeme ukazatel na paket o velikost IP hlavičky.

Ukazatel na paket je teď nastaven na začátek hlavičky TCP packetu. Zde můžeme nalézt port odesílatele a příjemce na kterém pracuje přenos a TCP Flags, které můžeme využít k odhalení ukončení TCP spojení. Nakonec opět zjistíme velikost TCP hlavičky(tentokrát uložená v horních 4 bitech 13. bytu a navíc uložena ve formátu počtu čtveřic bytů) a o tuto velikost(vynásobenou 4) inkrementujeme ukazatel na paket.

Konečně se nacházíme u samotného obsahu TCP packetu. Hned z druhého a třetího bytu můžeme zjistit, zda se jedná o TLS paket nebo ne. V případě že nejedná, dále nás obsah packetu nezajímá a můžeme posunout ukazatel na začátek dalšího pcapng bloku. Pokud se však o TLS paket jedná, nejprve zjistíme, typ obsahu toho packetu nacházející se na prvním bytu. Pokud se jedná o handshake a navíc typ Hello, můžeme zkontrolovat, zda již existuje spojení na těchto adresách a portech, a pokud neexistuje, založit nové. V ostatních případech stačí zjistit, zda se jedná o již probíhající spojení, a pokud tomu tak je, inkrementovat počet přijatých paketů a zvýšit hodnotu přijatých bytů o hodnotu payload.

Testování a zdroje

Nejprve jsem si otestoval, zda offline analýza odhalí proběhlou spolužákem vytvořenou SSL komunikaci, u níž lze jednoduše manuálně zkontrolovat počet paketů.

```
[louda@Arch13 ISA-SSL]$ ./sslsniff -r tests/seznam.pcapng
2020-10-07 13:37:36.000000,192.168.43.121,48102,77.75.75.176,seznam.cz,1761,33,0.0
[louda@Arch13 ISA-SSL]$
```

Poté co počet odpovídal jsem zapnul ve stejnou chvíli sniffer a odchyťování komunikace ve Wiresharku a vydal se navštívit stránku www.seznam.cz.

```
[louda@Arch13 ISA-SSL]$ sudo ./sslsniff -i wlp3s0
2020-11-18 22:50:54.1605736264,192.168.1.6,49146,77.75.78.55,login.seznam.cz,6609,25,10.10
^C
[louda@Arch13 ISA-SSL]$
```

Po zobrazení spojení jsem vypl zachytávání snifferu i Wiresharku, uložil ve Wiresharku zachycenou komunikaci a zkontroloval, zda odpovídá počet SSL spojení.

```
[louda@Arch13 ISA-SSL]$ ./sslsniff -r testingInProduction.pcapng
2020-11-18 22:52:00.000000,192.168.1.6,49146,77.75.78.55,login.seznam.cz,4119,24,0.0
[louda@Arch13 ISA-SSL]$
```

Bohužel jsem nedokázal odhalit důvod rozdílu v počtu paketů a počtu přenesných bytů komunikací.

Při práci na projektu jsem čerpal primárně ze zdroje popisujícího strukturu pcapng souboru^[1], označení hlaviček linkové vrstvy^[2], strukturu IPv4^[3] a IPv6^[4] paketů, a jednou z nejdůležitějších pomocí byl volně dostupný program wireshark^[5].

Dále jsem z veřejného zdroje převzal úryvek kódu načítající soubor do pole bytů.^[6]

U části projektu týkající se sniffingu jsem se inspiroval svým vlastním projektem z letního semestru druhého ročníku v předmětu IPK.

[1] <https://tools.ietf.org/id/draft-tuexen-opsawg-pcapng-01.html>

[2] <https://www.tcpdump.org/linktypes.html>

[3] <https://cs.wikipedia.org/wiki/IPv4>

[4] <https://cs.wikipedia.org/wiki/IPv6>

[5] <https://www.wireshark.org/>

[6] <https://stackoverflow.com/questions/22059189/read-a-file-as-byte-array>